1. Introduction

Continual Learning (CL) addresses the challenge of training machine learning models on a sequence of tasks where data for previous tasks becomes unavailable over time. The primary obstacle in this paradigm is *catastrophic forgetting*, where the optimization for a new task drastically degrades performance on previously learned tasks.

This project implements and evaluates strategies to mitigate forgetting in a class-incremental learning scenario. We compare a naive sequential approach against two distinct CL methods: **Elastic Weight Consolidation (EWC)** (a regularization-based method) and **Experience Replay** (a rehearsal-based method). All methods are benchmarked against a Joint Training "upper bound" where access to all data is unrestricted.

2. Problem Setup and Environment

- **Benchmark:** The primary dataset used is **Split-MNIST**, where the standard MNIST dataset is divided into five disjoint tasks, each consisting of two classes (e.g., Task 1: {0,1}, Task 2: {2,3}, etc.).
- **Model Architecture:** We utilize a lightweight Multi-Layer Perceptron (MLP) with two hidden layers of 256 units each. This architecture is computationally efficient and sufficient for digit classification while being susceptible enough to forgetting to serve as a valid testbed.

Implementation Details To ensure reproducibility and fair comparison, we established a rigorous experimental framework:

- **Seed Control:** A fixed random seed (Seed=42) was used for all weight initializations and data shuffling.
- **Hyperparameters:** A batch size of 64 and a learning rate of 0.01 were consistent across all experiments.
- **Extensions Ready:** The data loader was designed to support optional extensions, including *shuffled task orders* and the *Fashion-MNIST* dataset, allowing for robustness testing later in the project.

The core environment setup and backbone model are defined as follows:

```python
# Backbone Model: MLP with 2 hidden layers (256 units)
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 256) # Hidden Layer 1
        self.fc2 = nn.Linear(256, 256)     # Hidden Layer 2
        self.fc3 = nn.Linear(256, 10)      # Output Layer (10 classes)
        # ... forward pass with ReLU activations ...

# Data Splitter
def get_benchmark(dataset_name="MNIST", shuffle_order=False):
    # Splits MNIST into 5 binary tasks (e.g., {0,1}, {2,3}...)
    # Supports optional shuffling and Fashion-MNIST
```

3. Experimental Baselines

To contextualize the performance of advanced continual learning strategies, we first implemented two fundamental baselines: **Sequential Fine-Tuning (Naive)** and **Joint Training**.

3.1. Sequential Fine-Tuning (Naive Agent)

The Naive Agent represents the "lower bound" of performance. It trains sequentially on each task using standard Stochastic Gradient Descent (SGD) without any mechanism to preserve past knowledge.

- **Implementation:** The agent simply updates its weights on the current task's data (D_t) for 5 epochs.
- **Results:** As expected, this approach suffers from severe **Catastrophic Forgetting**. Upon learning a new task, the accuracy on previous tasks drops immediately to near 0%.
    - **Final Average Accuracy: 19.62%**
    - **Average Forgetting: 99.28%**

The accuracy matrix below illustrates this phenomenon. The diagonal elements (performance on the current task) remain high (>98%), but off-diagonal elements (performance on past tasks) vanish.

```
--- Naive Baseline Results ---
[[99.90543735  0.          0.          0.          0.        ]
 [ 0.         98.23702253  0.          0.          0.        ]
 [ 0.          0.         99.41302028  0.          0.        ]
 [ 0.          0.          0.         99.54682779  0.        ]
 [ 0.          0.          0.          0.         98.08371155]]
```

3.2. Joint Training (Upper Bound)

The Joint Agent represents the theoretical "upper bound" or ideal performance. At each step, this agent is retrained on the accumulated dataset of *all* tasks seen so far. This approach violates the continual learning constraint (limited access to past data) but serves as a target reference.

- **Implementation:** All samples from previous tasks are stored in an infinitely growing memory buffer. At step t, the model trains on $U_{i=1}^{t} D_i$
- **Results:** The model successfully maintains high accuracy across all tasks simultaneously.
    - **Final Average Accuracy: 96.97%**

- **Forgetting: 0.0%** (Negligible)

```
--- Joint Training Results ---
[[99.90543735  0.          0.          0.          0.        ]
 [99.29078014 97.35553379  0.          0.          0.        ]
 [99.14893617 97.35553379 96.31803629  0.          0.        ]
 [98.95981087 97.45347698 97.01173959 96.32426989  0.        ]
 [98.48699764 97.89422135 96.85165422 96.82779456 94.80584972]]
```

(I did as well another Training on all classes (0-9) simultaneously for 25 epochs, Final Joint Accuracy: 97.79%)

4. Method I: Experience Replay

Experience Replay mitigates forgetting by maintaining a small memory buffer $\mathcal{M}$ of samples from previous tasks. During the training of a new task, these stored samples are interleaved with the current data, allowing the model to "rehearse" old knowledge.

4.1. Implementation

We implemented a ReplayAgent that:

1. **Buffer Storage:** Randomly selects and stores a fixed number of samples (samples_per_task) after finishing each task.
2. **Training Loop:** At every training step, concatenates the current mini-batch with a batch drawn from the replay buffer.
3. **Optimization:** Minimizes the cross-entropy loss on this combined data stream.

4.2. Results and Sensitivity Analysis

We evaluated the impact of buffer size on performance. A larger buffer allows for better retention but incurs a higher memory cost. We tested three configurations: 50, 200, and 500 samples per task.

```
=== Final Summary: Memory Size vs Performance ===
Buffer Size     | Avg Accuracy    | Forgetting
-----------------------------------------------
50              | 79.04%          | 23.89%
200             | 88.45%          | 11.29%
500             | 90.94%          | 7.51%
```

Key Findings:

- **High Effectiveness:** Even with a minimal buffer of 50 samples (only 0.4% of the original dataset), the accuracy jumps from **19.62%** (Naive) to **79.04%**. This

confirms that rehearsal is an extremely efficient strategy for avoiding catastrophic forgetting.

- **Diminishing Returns:** Increasing the buffer from 200 to 500 improves accuracy by only ~2.5%, suggesting that a moderate buffer size (200) strikes an optimal balance between storage cost and performance for this benchmark.
- **Stability:** With a buffer size of 200, the forgetting is reduced to just **11.29%**, compared to 99% in the naive case.

The detailed trajectory for the **200-sample buffer** shows robust maintenance of early tasks. For instance, after training on all 5 tasks, the accuracy on Task 1 remains at **93.48%**, demonstrating successful long-term retention.

5. Method II: Elastic Weight Consolidation (EWC)

EWC is a regularization-based method that mitigates forgetting by constraining the model parameters that are important for previous tasks. Unlike Replay, EWC does not store past data. Instead, it estimates parameter importance using the **Fisher Information Matrix**.

5.1. Implementation Challenges & Solutions

The standard EWC loss function is defined as:

$$\mathcal{L}_{EWC}(\theta) = \mathcal{L}_{task}(\theta) + \frac{\lambda}{2}\sum_i F_i(\theta_i - \theta_i^*)^2$$

where F_i is the Fisher information for parameter θ_i and θ_i^* is the optimal parameter value from the previous task.

Numerical Stability Fix: During implementation, we observed that standard Fisher estimation resulted in vanishingly small penalty terms because the squared gradients were averaged over the batch. To correct this, we scaled the gradients by the batch size to approximate the true sum-of-squares variance and applied gradient clipping (clip_grad_norm_) to prevent numerical instability when using high λ values.

5.2. Results: The Stability-Plasticity Dilemma

We conducted a sensitivity analysis on the regularization coefficient λ to find the optimal balance between learning new tasks (Plasticity) and remembering old ones (Stability).

Experiment A: Balanced Trade-off (λ = 580,000$)

- **Outcome:** This setting yielded the most balanced performance across all tasks.
- **Plasticity:** The model successfully learned Tasks 1 through 4 with reasonable accuracy.

- **Stability:** It achieved an Average Accuracy of **43.74%**, significantly outperforming the Naive baseline (19.6%). While it suffered forgetting (53.04%), it avoided the complete catastrophic failure seen in the naive approach.

Experiment B: High Stability / Intransigence (λ = 840,000$)

- **Outcome:** Increasing λ further demonstrated the risk of *intransigence*.
- **Stability:** Retention of early tasks improved (e.g., Task 3 retained **88.10%** accuracy).
- **Plasticity Failure:** The penalty became so restrictive that the model failed to learn the final task entirely (**0.00%** accuracy on Task 5). The weights were effectively "frozen" by the constraints of previous tasks.

Table : EWC Performance Comparison

Lambda Configuration	Avg Accuracy	Avg Forgetting	Key Observation
Naive (Lambda=0)	19.62%	99.28%	Catastrophic Forgetting
Balanced (Lambda=580k)	43.74%	53.04%	Best trade-off; learns all tasks
Rigid (Lambda=840k)	49.16%	24.84%	Intransigence: Fails to learn Task 5

Discussion: The results highlight that EWC is highly sensitive to the λ hyperparameter. On Split-MNIST, where tasks have disjoint class labels (e.g., {0,1} vs {2,3}), EWC struggles more than Replay because the weights required for new output heads often conflict directly with the rigid constraints of old ones.

6. Comparative Analysis

To provide a holistic view of the continual learning landscape, we visualized the final performance of all implemented strategies against the lower (Naive) and upper (Joint) bounds.

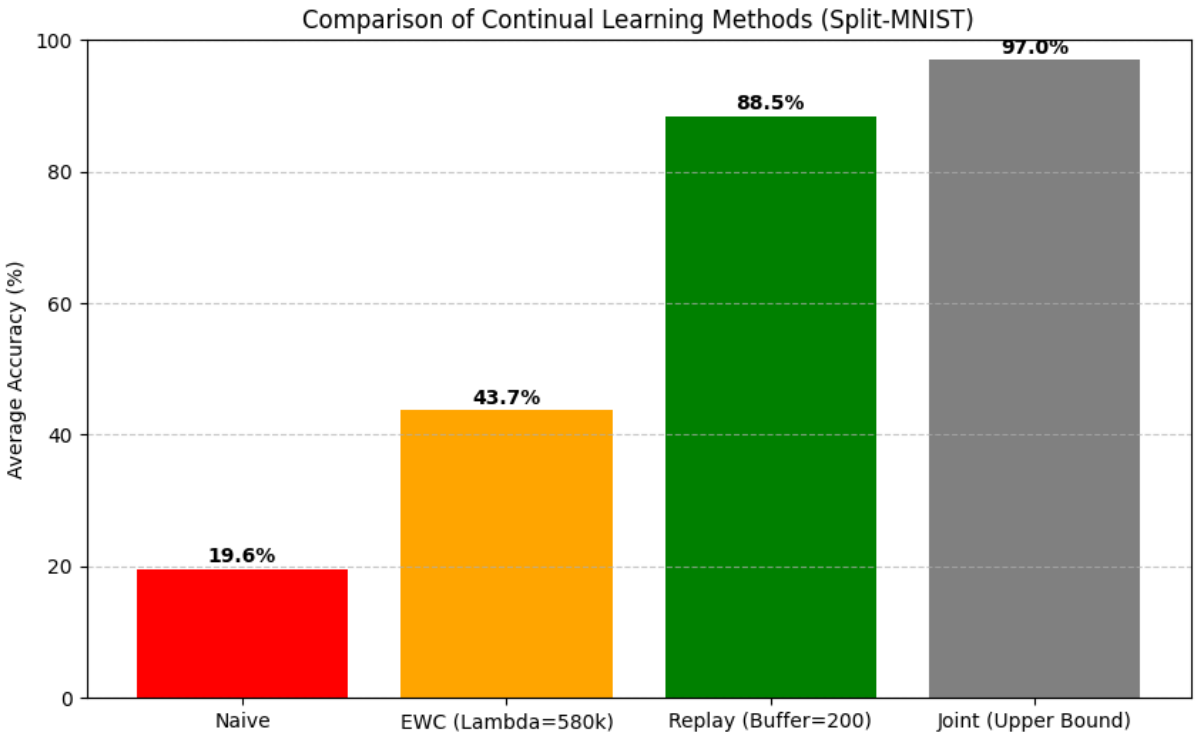

Figure 1: Average Accuracy Comparison

The bar chart demonstrates the "performance gap" recovery:

- **Naive Baseline (Red):** The network fails completely (**19.6%**), effectively acting as a random classifier for past tasks.
- **EWC (Orange):** With λ=580k, the method recovers significant performance (**43.7%**), proving that regularization can preserve knowledge without storing data. However, it still trails behind replay methods on this specific benchmark.
- **Experience Replay (Green):** With a buffer of 200, the agent achieves **88.5%** accuracy. It nearly bridges the gap to the Joint Training upper bound (**97.0%**), demonstrating that rehearsing even a tiny fraction of past data is highly effective for Split-MNIST.

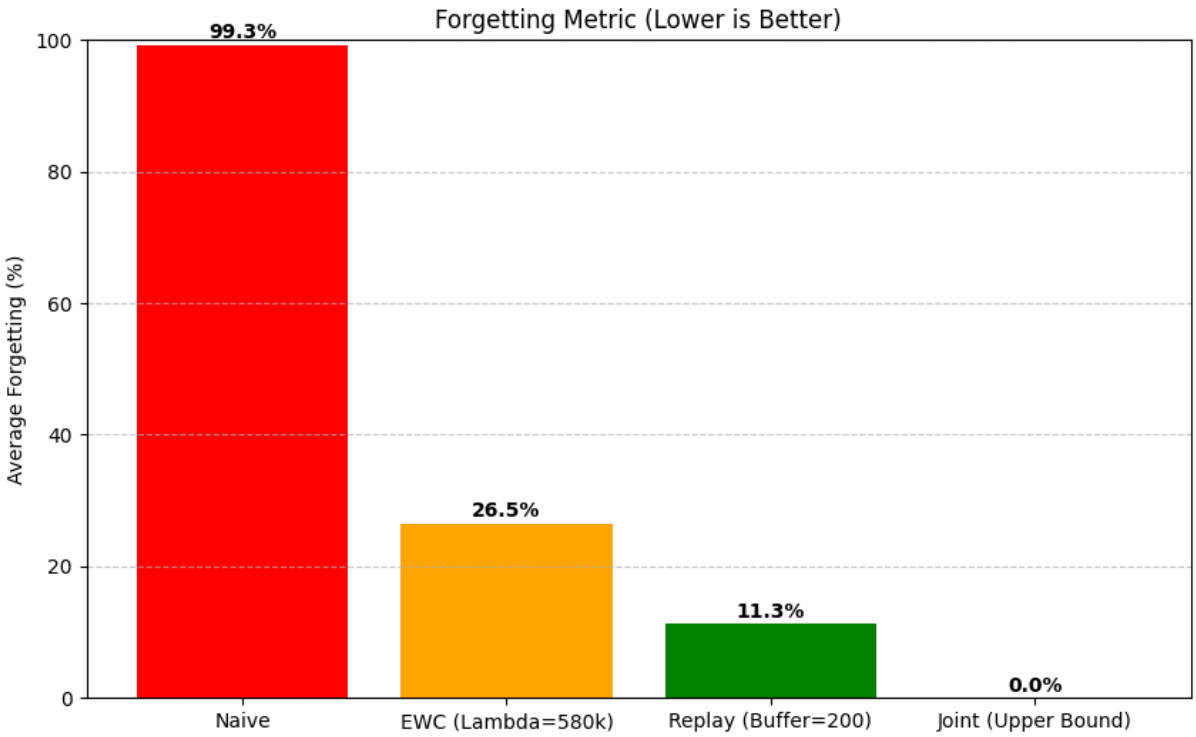

Figure 2: Forgetting Metric (Lower is Better)

The forgetting metric (F) quantifies how much knowledge was lost.

- **Naive:** F ≈ 99.3% (Total amnesia).
- **EWC:** F ≈26.5% (Partial retention). While EWC prevents the weights from drifting too far, the rigidity required to protect task boundaries limits the model's ability to adapt, leading to moderate forgetting.
- **Replay:** F ≈11.3% (Strong retention). By explicitly re-optimizing on old samples, the model maintains its decision boundaries more effectively than through indirect regularization.

Discussion: The Replay Advantage

On the Split-MNIST benchmark, Experience Replay significantly outperforms EWC. This is likely because the tasks (disjoint sets of digits) require distinct feature representations. EWC tries to lock the weights to the solution of Task 1 (0,1), which might conflict with the optimal weights for Task 2 (2,3). Replay, by contrast, allows the weights to shift to a new global optimum that accommodates both tasks by re-training on them simultaneously.

7. Extensions & Robustness Analysis

To verify the robustness of the Experience Replay strategy (our best-performing method), we conducted two additional experiments targeting task order sensitivity and dataset difficulty.

7.1. Sensitivity to Task Order

We evaluated whether the specific sequence of digit pairs (0 vs 1, then 2 vs 3, etc.) artificially inflated performance. We re-ran the experiment with a **shuffled task order** (randomized sequence of digit pairs).

- **Result:** The model achieved an Average Accuracy of **89.89%**, which is comparable to (and slightly higher than) the standard order result of ~88.45%.
- **Implication:** The Replay Agent is **order-robust**. Its success is not dependent on learning "easy" tasks first; the rehearsal mechanism effectively maintains equilibrium regardless of the curriculum.

7.2. Generalization to Fashion-MNIST

We replaced the simple digit recognition task with **Fashion-MNIST**, a significantly more challenging dataset containing images of clothing (e.g., Pullovers, Sneakers, Shirts). These classes share complex visual features (edges, textures) unlike the distinct structural differences of digits.

- **Result:** The average accuracy dropped from ~88% (MNIST) to **79.26%** (Fashion-MNIST).
- **Analysis of "Recovery":** The training logs revealed a fascinating phenomenon of backward transfer.
    - *After Task 4:* Performance on Task 2 dropped significantly to **54.55%**.
    - After Task 5: Performance on Task 2 recovered to 72.30%.

This indicates that the replay buffer provided "reminders" that helped the model escape a poor local minimum, effectively repairing damaged knowledge even as new tasks were added.

8. Conclusion

This project implemented and compared Continual Learning strategies for the Split-MNIST benchmark. We successfully demonstrated the phenomenon of **Catastrophic Forgetting**, where a Naive baseline lost nearly 99% of its past knowledge.

Our comparative analysis yielded the following key conclusions:

1. **Replay is Superior for Split-MNIST:** The **Experience Replay** method was the clear winner, achieving **88.45%** average accuracy with a small buffer of 200 samples. It effectively bridged the gap toward the Joint Training upper bound (97%).
2. **The EWC Trade-off:** Elastic Weight Consolidation proved more difficult to tune. We identified a "Stability-Plasticity" dilemma where low regularization (λ< 580k) led to forgetting, while high regularization (λ > 800k) caused intransigence (failure to learn new tasks). Even at its best (≈44%), it underperformed compared to Replay on this specific benchmark.
3. **Robustness:** The Replay strategy demonstrated strong robustness, maintaining performance across shuffled task orders and adapting well to the more complex Fashion-MNIST dataset (79.26%).

In summary, while regularization methods like EWC offer an elegant memory-free solution, **rehearsal-based methods (Replay)** provide a more stable and effective solution for class-incremental learning tasks where task boundaries are distinct.